

Chapter 6. IP and DNS

Every computer that is connected to a network needs an IP address. In [Chapter 1](#), *Network Programming and Python*, an introduction to TCP/IP networking was presented. The IP address labels a machine's network interface with a numeric identifier, which also identifies the location of the machine, albeit with limited reliability. **Domain Name System (DNS)** is a core network service that maps the names to the IP addresses and vice-versa. In this chapter, we will mainly focus on manipulating the IP and DNS protocols with the help of Python. In addition to this, we will briefly discuss the **Network Time Protocol (NTP)**, which helps in synchronizing the time with a centralized time server. The following topics will be discussed here:

- Retrieving the network configuration of a local machine
- Manipulating the IP addresses
- The GeoIP look-ups
- Working with DNS
- Working with NTP

Retrieving the network configuration of a local machine

Before doing anything else, let's ask in the Python language, *What's my name?*. In networking terms, this is equivalent to finding out the machine's name or the host's name. On the shell command-line, this can be discovered by using the `hostname` command. In Python, you can do this by using the `socket` module.

```
>>> import socket
>>> socket.gethostname()
'debian6box.localdomain.loc'
```

Now, we would like to see the local machine IP. This can be seen by using the `ifconfig` command in Linux and by using the `ipconfig` command in the Windows OS. But, we'd like to do this in Python by using the following built-in function:

```
>>> socket.gethostbyname('debian6box.localdomain.loc')
'10.0.2.15'
```

As you can see, this is the IP of the first network interface. It can also show us the IP of the loopback interface (127.0.0.1) if your DNS or hostfile has not been configured properly. In Linux/UNIX, the following line can be added to your `/etc/hosts` file for obtaining the correct IP address:

```
10.0.2.15          debian6box.localdomain.loc      debian6box
```

This process is known as a host-file based name resolution. You can send a query to a DNS server and ask for the IP address of a specific host. If the name has been registered properly, then you will get a response from the server. But, before making a query to the remote server, let us first discover some more information about the network interface and the gateway machine of your network.

In every LAN, a host is configured to act as a gateway, which talks to the *outside* world. In order to find the network address and the netmask, we can use the Python third-party library `netifaces` (version > 0.10.0). This will pull all the relevant information. For example, you can call `netifaces.gateways()` for finding the gateways that are configured to the outside world. Similarly, you can enumerate the network interfaces by calling `netifaces.interfaces()`. If you would like to know all the IP addresses of a particular interface `eth0`, then you can call `netifaces.ifaddresses('eth0')`. The following code listing shows the way in which you can list all the gateways and IP addresses of a local machine:

```
#!/usr/bin/env python
import socket
import netifaces

if __name__ == '__main__':
    # Find host info
    host_name = socket.gethostname()
```

```

ip_address = socket.gethostbyname(host_name)
print("Host name: {0}".format(host_name))

# Get interfaces list
ifaces = netifaces.interfaces()
for iface in ifaces:
    ipaddrs = netifaces.ifaddresses(iface)
    if netifaces.AF_INET in ipaddrs:
        ipaddr_desc = ipaddrs[netifaces.AF_INET]
        ipaddr_desc = ipaddr_desc[0]
        print("Network interface: {0}".format(iface))
        print("\tIP address: {0}".format(ipaddr_desc['addr']))
        print("\tNetmask: {0}".format(ipaddr_desc['netmask']))
# Find the gateway
gateways = netifaces.gateways()
print("Default gateway: {0}".format(gateways['default']
[netifaces.AF_INET][0]))

```

If you run this code, then this will print a summary of the local network configuration, which will be similar to the following:

```

$ python 6_1_local_network_config.py
Host name: debian6box
Network interface: lo
  IP address: 127.0.0.1
  Netmask: 255.0.0.0
Network interface: eth0
  IP address: 10.0.2.15
  Netmask: 255.255.255.0
Default gateway: 10.0.2.2

```


Manipulating IP addresses

Often you will need to manipulate IP addresses and perform some sort of operations on them. Python3 has a built-in `ipaddress` module to help you in carrying out this task. It has convenient functions for defining the IP addresses and the IP networks and for finding lots of useful information. For example, if you would like to know how many IP addresses exist in a given subnet, for instance, `10.0.1.0/255.255.255.0` or `10.0.2.0/24`, then you can find them with the help of the code snippet shown here. This module will provide several classes and factory functions; for example, the IP address and the IP network has separate classes. Each class has a variant for both IP version 4 (IPv4) and IP version 6 (IPv6). Some of the features have been demonstrated in the following section:

IP network objects

Let us import the `ipaddress` module and define a `net4` network.

```
>>> import ipaddress as ip
>>> net4 = ip.ip_network('10.0.1.0/24')
```

Now, we can find some useful information, such as `netmask`, the network/broadcast address, and so on, of `net4`:

```
>>> net4.netmask
IP4Address(255.255.255.0)
```

The `netmask` properties of `net4` will be displayed as an `IP4Address` object. If you are looking for its string representation, then you can call the `str()` method, as shown here:

```
>>> str(net4.netmask)
'255.255.255.0'
```

Similarly, you can find the network and the broadcast addresses of `net4`, by doing the following:

```
>>> str(net4.network_address)
10.0.1.0
>>> str(net4.broadcast_address)
10.0.1.255
```

How many addresses does `net4` hold in total? This can be found by using the command shown here:

```
>>> net4.num_addresses
256
```

So, if we subtract the network and the broadcast addresses, then the total available IP addresses will be 254. We can call the `hosts()` method on the `net4` object. It will produce a Python generator, which will supply all the hosts as `IPv4Address` objects.

```
>>> all_hosts = list(net4.hosts())
>>> len(all_hosts)
254
```

You can access the individual IP addresses by following the standard Python list access notation. For example, the first IP address would be the following:

```
>>> all_hosts[0]
IPv4Address('10.0.1.1')
```

You can access the last IP address by using the list notation for accessing the last item of a list, as shown here:

```
>>> all_hosts[-1]
IPv4Address('10.0.1.1')
```

We can also find the subnet information from the `IPv4Network` objects, as follows:

```
>>> subnets = list( net4.subnets())
```

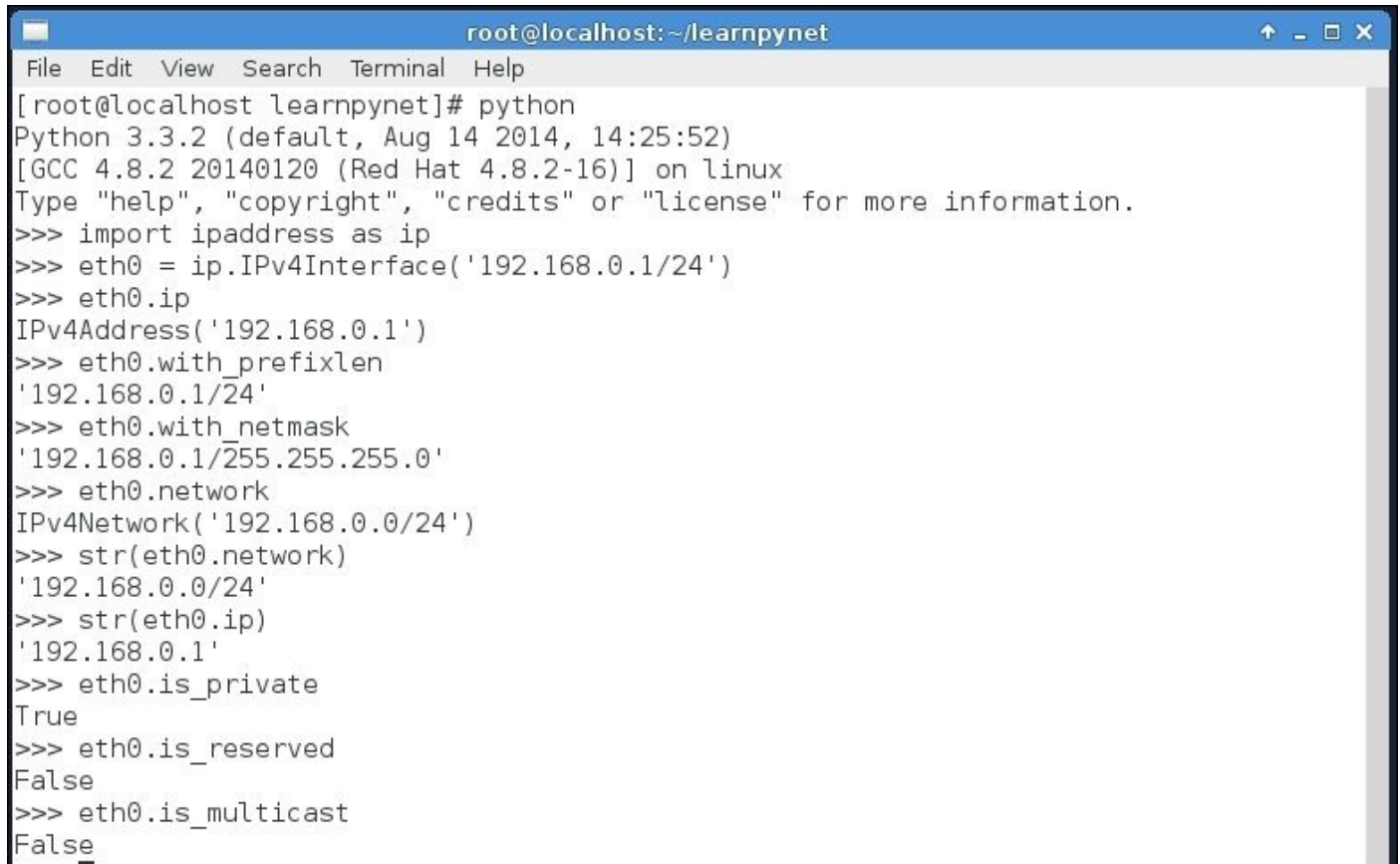
```
>>> subnets
[ IPv4Network('10.0.1.0/25'), IPv4Network('10.0.1.128/25') ]
```

Any IPv4Network object can tell about its parent supernet, which is the opposite of the subnet.

```
>>> net4.supernet()
IPv4Network('10.0.1.0/23')
```

Network interface objects

In the `ipaddress` module, a convenient class is used for representing an interface's IP configuration in detail. The IPv4 Interface class takes an arbitrary address and behaves like a network address object. Let us define and discuss our network interface `eth0`, as shown in following screenshot:

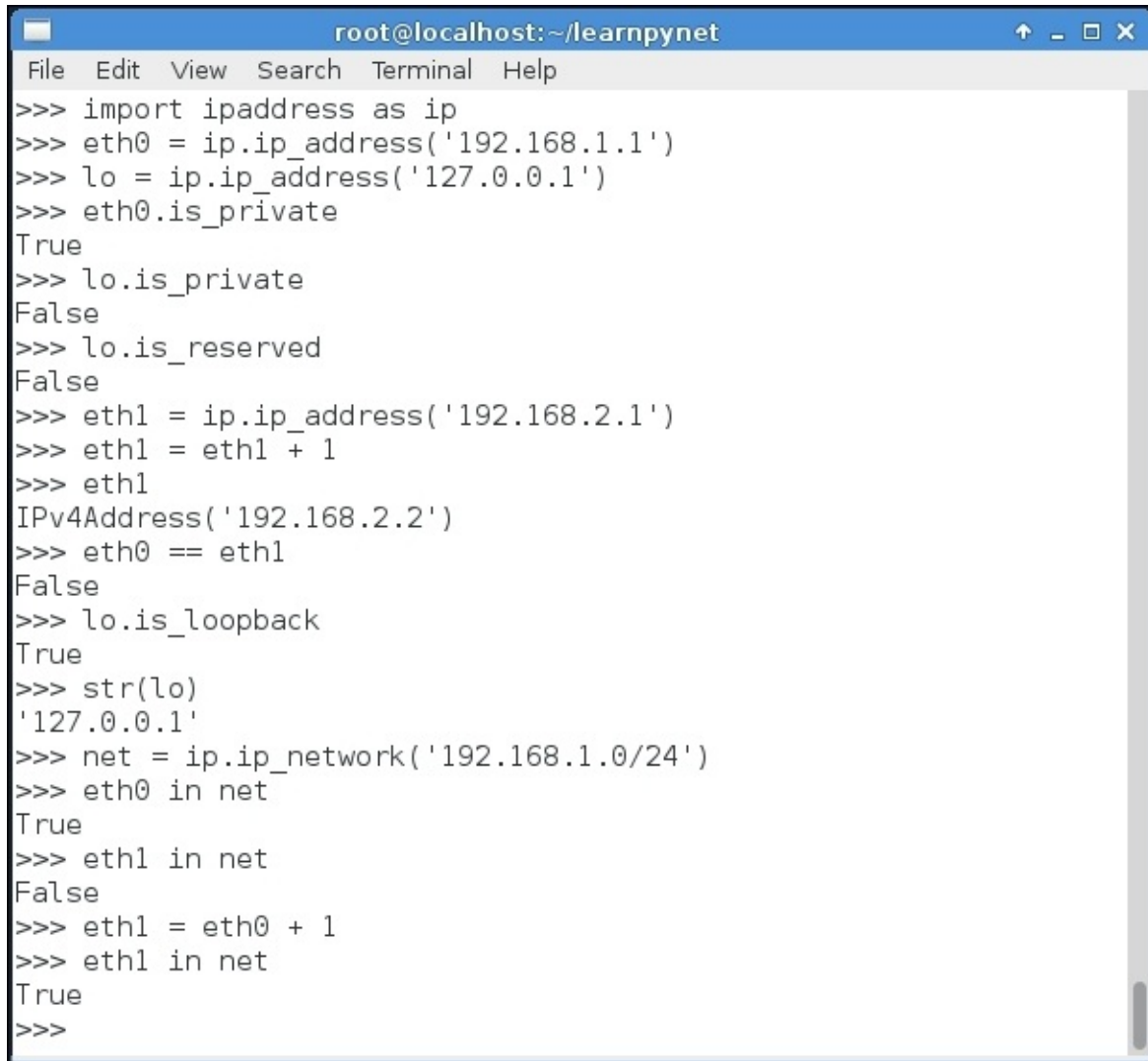
A screenshot of a terminal window titled 'root@localhost: ~/learnpynet'. The terminal shows a Python session where an IPv4Interface object named 'eth0' is created with the address '192.168.0.1/24'. Various attributes of the object are accessed, including the IP address, network address, and flags like 'is_private', 'is_reserved', and 'is_multicast'.

```
root@localhost: ~/learnpynet
File Edit View Search Terminal Help
[root@localhost learnpynet]# python
Python 3.3.2 (default, Aug 14 2014, 14:25:52)
[GCC 4.8.2 20140120 (Red Hat 4.8.2-16)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import ipaddress as ip
>>> eth0 = ip.IPv4Interface('192.168.0.1/24')
>>> eth0.ip
IPv4Address('192.168.0.1')
>>> eth0.with_prefixlen
'192.168.0.1/24'
>>> eth0.with_netmask
'192.168.0.1/255.255.255.0'
>>> eth0.network
IPv4Network('192.168.0.0/24')
>>> str(eth0.network)
'192.168.0.0/24'
>>> str(eth0.ip)
'192.168.0.1'
>>> eth0.is_private
True
>>> eth0.is_reserved
False
>>> eth0.is_multicast
False
```

As you can see in the preceding screenshot, a network interface `eth0` with the `IPv4Address` class has been defined. It has some interesting properties, such as IP, network address, and so on. In the same way as with the network objects, you can check if the address is private, reserved, or multicast. These address ranges have been defined in various RFC documents. The `ipaddress` module's help page will show you the links to those RFC documents. You can search this information in other places as well.

The IP address objects

The IP address classes have many more interesting properties. You can perform some arithmetic and logical operations on those objects. For example, if an IP address is greater than another IP address, then you can add numbers to the IP address objects, and this will give you a corresponding IP address. Let's see a demonstration of this in the following screenshot:

A screenshot of a terminal window titled 'root@localhost: ~/learnpynet'. The terminal shows a series of Python commands and their outputs. The commands include importing the 'ipaddress' module, creating IP address objects for 'eth0' (192.168.1.1), 'lo' (127.0.0.1), and 'eth1' (192.168.2.1). It then tests various properties like 'is_private', 'is_reserved', 'is_loopback', and 'is_loopback'. It also demonstrates arithmetic operations, such as adding 1 to 'eth1' to get '192.168.2.2', and checking if an IP address belongs to a specific network ('192.168.1.0/24').

```
root@localhost: ~/learnpynet
File Edit View Search Terminal Help
>>> import ipaddress as ip
>>> eth0 = ip.ip_address('192.168.1.1')
>>> lo = ip.ip_address('127.0.0.1')
>>> eth0.is_private
True
>>> lo.is_private
False
>>> lo.is_reserved
False
>>> eth1 = ip.ip_address('192.168.2.1')
>>> eth1 = eth1 + 1
>>> eth1
IPv4Address('192.168.2.2')
>>> eth0 == eth1
False
>>> lo.is_loopback
True
>>> str(lo)
'127.0.0.1'
>>> net = ip.ip_network('192.168.1.0/24')
>>> eth0 in net
True
>>> eth1 in net
False
>>> eth1 = eth0 + 1
>>> eth1 in net
True
>>>
```

Demonstration of the ipaddress module

Here, the `eth0` interface has been defined with a private IP address, which is `192.168.1.1`, and `eth1` has been defined with another private IP address, which is `192.168.2.1`. Similarly the loopback interface `lo` is defined with IP address `127.0.0.1`. As you can see, you can add numbers to the IP address and it will give you the next IP address with the same sequence.

You can check if an IP is a part of a specific network. Here, a network `net` has been defined by the network address, which is `192.168.1.0/24`, and the membership of `eth0` and `eth1` has been tested against that. A few other interesting properties, such as `is_loopback`, `is_private`, and so on, have also been tested here.

Planning IP addresses for your local area network

If you are wondering how to pick-up a suitable IP subnet, then you can experiment with the `ipaddress` module. The following code snippet will show an example of how to choose a specific subnet, based on the number of necessary host IP addresses for a small private network:

```
#!/usr/bin/env python
import ipaddress as ip

CLASS_C_ADDR = '192.168.0.0'

if __name__ == '__main__':
    not_configed = True
    while not_configed:
        prefix = input("Enter the prefixlen (24-30): ")
        prefix = int(prefix)
        if prefix not in range(23, 31):
            raise Exception("Prefixlen must be between 24 and 30")
        net_addr = CLASS_C_ADDR + '/' + str(prefix)
        print("Using network address:%s " %net_addr)
        try:
            network = ip.ip_network(net_addr)
        except:
            raise Exception("Failed to create network object")
        print("This prefix will give %s IP addresses" %
(network.num_addresses))
        print("The network configuration will be")
        print("\t network address: %s" %str(network.network_address))
        print("\t netmask: %s" %str(network.netmask))
        print("\t broadcast address: %s" %str(network.broadcast_address))
        first_ip, last_ip = list(network.hosts())[0], list(network.hosts())
[-1]
        print("\t host IP addresses: from %s to %s" %(first_ip, last_ip))
        ok = input("Is this configuration OK [y/n]? ")
        ok = ok.lower()
        if ok.strip() == 'y':
            not_configed = False
```

If you run this script, then it will show an output similar to the following:

```
# python 6_2_net_ip_planner.py
Enter the prefixlen (24-30): 28
Using network address:192.168.0.0/28
This prefix will give 16 IP addresses
The network configuration will be
    network address: 192.168.0.0
    netmask: 255.255.255.240
    broadcast address: 192.168.0.15
    host IP addresses: from 192.168.0.1 to 192.168.0.14
Is this configuration OK [y/n]? n
Enter the prefixlen (24-30): 26
Using network address:192.168.0.0/26
This prefix will give 64 IP addresses
The network configuration will be
```

```
network address: 192.168.0.0
netmask: 255.255.255.192
broadcast address: 192.168.0.63
host IP addresses: from 192.168.0.1 to 192.168.0.62
Is this configuration OK [y/n]? y
```


GeoIP look-ups

At times, it will be necessary for many applications to look-up the location of the IP addresses. For example, many website owners can be interested in tracking the location of their visitors and in classifying their IPs according to criteria, such as country, city, and so on. There is a third-party library called **python-geoip**, which has a robust interface for giving you the answer to your IP location query. This library is provided by MaxMind, which also provides the option for shipping a recent version of the Geolite2 database as the python-geoip-geolite2 package. This includes the GeoLite2 data created by MaxMind, which is available at www.maxmind.com under the creative commons Attribution-ShareAlike 3.0 Unported License. You can also buy a commercial license from their website.

Let's see an example of how to use this Geo-lookup library.:

```
import socket
from geoip import geolite2
import argparse

if __name__ == '__main__':
    # Setup commandline arguments
    parser = argparse.ArgumentParser(description='Get IP Geolocation info')
    parser.add_argument('--hostname', action="store", dest="hostname",
                        required=True)

    # Parse arguments
    given_args = parser.parse_args()
    hostname = given_args.hostname
    ip_address = socket.gethostbyname(hostname)
    print("IP address: {0}".format(ip_address))

    match = geolite2.lookup(ip_address)
    if match is not None:
        print('Country: ', match.country)
        print('Continent: ', match.continent)
        print('Time zone: ', match.timezone)
```

This script will show an output similar to the following:

```
$ python 6_3_geoip_lookup.py --hostname=amazon.co.uk
IP address: 178.236.6.251
Country: IE
Continent: EU
Time zone: Europe/Dublin
```

You can find more information about this package from the developer's website, which is at <http://pythonhosted.org/python-geoip/>.

DNS look-ups

The IP address can be translated into human readable strings called domain names. DNS is a big topic in the world of networking. In this section, we will create a DNS client in Python, and see how this client will talk to the server by using Wireshark.

A few DNS client libraries are available from PyPI. We will focus on the `dnspython` library, which is available at <http://www.dnspython.org/>. You can install this library by using either the `easy_install` command or the `pip` command:

```
$ pip install dnspython
```

Making a simple query regarding the IP address of a host is very simple. You can use the `dns.resolver` submodule, as follows:

```
import dns.resolver
answers = dns.resolver.query('python.org', 'A')
for rdata in answers:
    print('IP', rdata.to_text())
```

If you want to make a reverse look-up, then you need to use the `dns.reversename` submodule, as shown here:

```
import dns.reversename
name = dns.reversename.from_address("127.0.0.1")
print name
print dns.reversename.to_address(name)
```

Now, let's create an interactive DNS client script that will do a complete look-up of the possible records, as shown here:

```
import dns.resolver

if __name__ == '__main__':
    lookup_continue = True
    while lookup_continue:
        name = input('Enter the DNS name to resolve: ')
        record_type = input('Enter the query type [A/MX/CNAME]: ')
        answers = dns.resolver.query(name, record_type)
        if record_type == 'A':
            print('Got answer IP address: %s' % [x.to_text() for x in
answers])
        elif record_type == 'CNAME':
            print('Got answer Aliases: %s' % [x.to_text() for x in answers])
        elif record_type == 'MX':
            for rdata in answers:
                print('Got answers for Mail server records:')
                print('Mailserver', rdata.exchange.to_text(), 'has
preference', rdata.preference)
            print('Record type: %s is not implemented' % record_type)
        lookup_more = input("Do you want to lookup more records? [y/n]: ")
        if lookup_more.lower() == 'n':
            lookup_continue = False
```

If you run this script with some input, then you will have an output similar to the

following:

```
$ python 6_4_dns_client.py
Enter the DNS name to resolve: google.com
Enter the query type [A/MX/CNAME]: MX
Got answers for Mail server records:
Mailserver alt4.aspmx.l.google.com. has preference 50
Got answers for Mail server records:
Mailserver alt2.aspmx.l.google.com. has preference 30
Got answers for Mail server records:
Mailserver alt3.aspmx.l.google.com. has preference 40
Got answers for Mail server records:
Mailserver aspmx.l.google.com. has preference 10
Got answers for Mail server records:
Mailserver alt1.aspmx.l.google.com. has preference 20
Do you want to lookup more records? [y/n]: y
Enter the DNS name to resolve: www.python.org
Enter the query type [A/MX/CNAME]: A
Got answer IP address: ['185.31.18.223']
Do you want to lookup more records? [y/n]: y
Enter the DNS name to resolve: pypi.python.org
Enter the query type [A/MX/CNAME]: CNAME
Got answer Aliases: ['python.map.fastly.net.']
Do you want to lookup more records? [y/n]: n
```

Inspecting DNS client/server communication

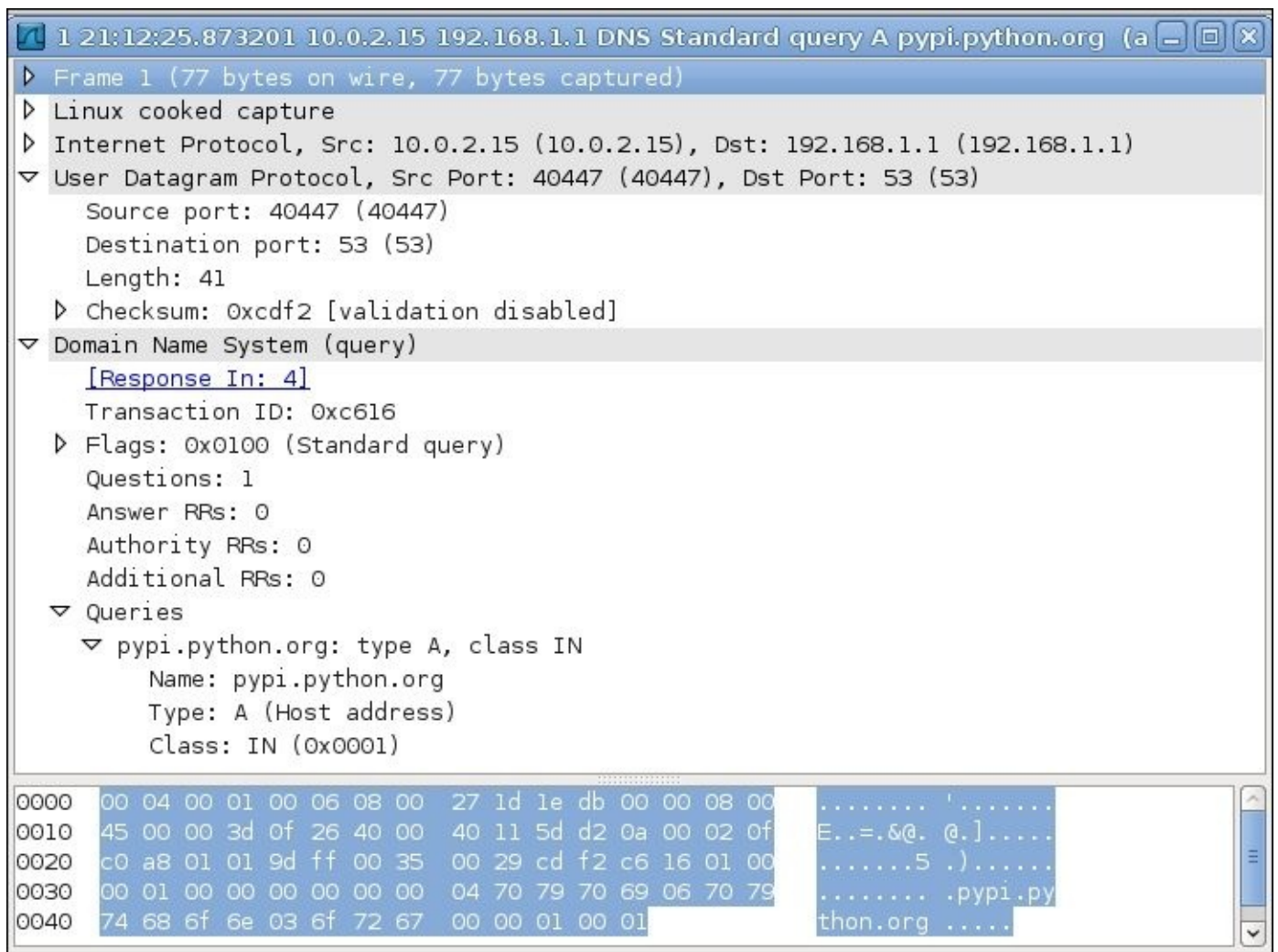
In previous chapters, perhaps you noticed how we captured network packets between the client and the server by using Wireshark. Here is an example of the session capturing, while a Python package was being installed from PyPI:

Source	Destination	Protocol	Info
10.0.2.15	192.168.1.1	DNS	Standard query A pypi.python.org
10.0.2.15	192.168.1.1	DNS	Standard query AAAA pypi.python.org
192.168.1.1	10.0.2.15	DNS	Standard query response CNAME python.map.fastly.net
192.168.1.1	10.0.2.15	DNS	Standard query response CNAME python.map.fastly.net A 185.31.19.223
10.0.2.15	192.168.1.1	DNS	Standard query A pypi.python.org
10.0.2.15	192.168.1.1	DNS	Standard query AAAA pypi.python.org
192.168.1.1	10.0.2.15	DNS	Standard query response CNAME python.map.fastly.net A 185.31.19.223
192.168.1.1	10.0.2.15	DNS	Standard query response CNAME python.map.fastly.net
10.0.2.15	192.168.1.1	DNS	Standard query A pypi.python.org
192.168.1.1	10.0.2.15	DNS	Standard query response CNAME python.map.fastly.net A 185.31.19.223
10.0.2.15	192.168.1.1	DNS	Standard query AAAA pypi.python.org
192.168.1.1	10.0.2.15	DNS	Standard query response CNAME python.map.fastly.net
10.0.2.15	192.168.1.1	DNS	Standard query A pypi.python.org
10.0.2.15	192.168.1.1	DNS	Standard query AAAA pypi.python.org
192.168.1.1	10.0.2.15	DNS	Standard query response CNAME python.map.fastly.net A 185.31.19.223
192.168.1.1	10.0.2.15	DNS	Standard query response CNAME python.map.fastly.net
10.0.2.15	192.168.1.1	DNS	Standard query A pypi.python.org
10.0.2.15	192.168.1.1	DNS	Standard query AAAA pypi.python.org
192.168.1.1	10.0.2.15	DNS	Standard query response CNAME python.map.fastly.net A 185.31.19.223
192.168.1.1	10.0.2.15	DNS	Standard query response CNAME python.map.fastly.net
10.0.2.15	192.168.1.1	DNS	Standard query A pypi.python.org
10.0.2.15	192.168.1.1	DNS	Standard query AAAA pypi.python.org
192.168.1.1	10.0.2.15	DNS	Standard query response CNAME python.map.fastly.net A 185.31.19.223
192.168.1.1	10.0.2.15	DNS	Standard query response CNAME python.map.fastly.net

FDNS client/server communication

In Wireshark you can specify port 53 by navigating to **Capture | Options | Capture filter**. This will capture all the DNS packets that were sent to/from your machine.

As you can see in the following screenshot, the client and the server have several request/response cycles the DNS records. It was started with a standard request for the host's address (A) and it was followed by a suitable response.



If you look deep inside a packet, then you can see the request format of the response from the server, as shown in the following screenshot:

4 21:12:25.910616 192.168.1.1 10.0.2.15 DNS Standard query response CNAME python.map

- Answers
 - pypi.python.org: type CNAME, class IN, cname python.map.fastly.net
 - Name: pypi.python.org
 - Type: CNAME (Canonical name for an alias)
 - Class: IN (0x0001)
 - Time to live: 23 hours, 11 minutes, 9 seconds
 - Data length: 23
 - Primary name: python.map.fastly.net
 - python.map.fastly.net: type A, class IN, addr 185.31.19.223
 - Name: python.map.fastly.net
 - Type: A (Host address)
 - Class: IN (0x0001)
 - Time to live: 20 seconds
 - Data length: 4
 - Addr: 185.31.19.223
- Authoritative nameservers
 - fastly.net: type NS, class IN, ns ns2.p04.dynect.net
 - fastly.net: type NS, class IN, ns ns4.p04.dynect.net
 - fastly.net: type NS, class IN, ns ns3.p04.dynect.net
 - fastly.net: type NS, class IN, ns ns1.p04.dynect.net
- Additional records
 - ns1.p04.dynect.net: type A, class IN, addr 208.78.70.4
 - ns2.p04.dynect.net: type A, class IN, addr 204.13.250.4
 - ns3.p04.dynect.net: type A, class IN, addr 208.78.71.4
 - ns4.p04.dynect.net: type A, class IN, addr 204.13.251.4

0000	00 00 00 01 00 06 52 54 00 12 35 02 00 00 08 00	RT ..5.....
0010	45 00 01 03 0e a7 00 00 40 11 9d 8b c0 a8 01 01	E..... @.....
0020	0a 00 02 0f 00 35 9d ff 00 ef 70 5b c6 16 81 80	5.. ..p[....
0030	00 01 00 02 00 04 00 04 04 70 79 70 69 06 70 79	pypi.py

